

Chapter 51 — Combinatorics / Counting

0. What You Will Learn in This Chapter

Combinatorics is the study of counting.

In interviews, combinatorics appears when you need to count:

- ways to arrange objects
- ways to choose objects
- paths in a grid
- subsets
- permutations
- strings under constraints
- handshakes/pairs
- distributions
- valid structures
- possible outcomes
- count modulo a large prime

This chapter is especially important for DE Shaw-level prep because DE Shaw-style rounds may include counting, probability, and expectation questions.

By the end of this chapter, you should understand:

- Counting principle
- Permutations
- Combinations
- Factorials
- nCr
- Pascal's triangle
- Modular combinations
- Precomputed factorials
- Modular inverse
- Inclusion-exclusion
- Stars and bars preview
- Catalan numbers
- Derangements preview
- Counting grid paths
- Counting subsets
- Counting with duplicates
- Common combinatorics interview patterns

Core problems covered in this chapter:

1. Pascal's Triangle
2. Pascal's Triangle II
3. Unique Paths, combinatorics solution
4. Count Sorted Vowel Strings
5. Climbing Stairs, counting view
6. Number of Ways to Reorder Array to Get Same BST
7. Count Vowels Permutation, counting DP view
8. Count Good Meals
9. Count Nice Pairs in an Array

- 10. Count Number of Teams
 - 11. Handshakes That Don't Cross, Catalan preview
 - 12. Derangements preview
 - 13. Combinations modulo prime
 - 14. Inclusion-exclusion examples
 - 15. Stars and bars examples
 - 16. Count ways with duplicates
-

1. What Is Combinatorics?

Combinatorics is about counting possibilities without listing them all.

Example:

How many ways to choose 2 people from 5?

People:

A, B, C, D, E

Pairs:

AB, AC, AD, AE, BC, BD, BE, CD, CE, DE

Answer:

10

Instead of listing, use formula:

$C(5, 2) = 10$

2. Why Counting Matters

Counting problems often have huge answer spaces.

Example:

How many paths exist from top-left to bottom-right in a 100x100 grid?

You cannot enumerate paths.

But combinatorics gives formula.

Counting is also foundational for probability:

$\text{probability} = \text{favorable outcomes} / \text{total outcomes}$

3. Fundamental Counting Principle

If one event can happen in a ways, and another independent event can happen in b ways, then together they can happen in:

$a * b$

Example:

You have:

3 shirts
2 pants

Outfits:

$$3 * 2 = 6$$

If there are multiple stages:

$$\text{choices} = \text{choices1} * \text{choices2} * \text{choices3} * \dots$$

4. Addition Rule

If two choices are mutually exclusive, add counts.

Example:

Choose either:

1 fruit from 3 fruits

OR

1 drink from 2 drinks

Total:

$$3 + 2 = 5$$

Use multiplication for independent stages.

Use addition for either/or cases.

5. Factorial

Factorial counts ways to arrange n distinct objects.

$$n! = n * (n - 1) * \dots * 1$$

Examples:

$$3! = 6$$

$$4! = 24$$

By definition:

$$0! = 1$$

Why?

There is exactly one way to arrange nothing.

6. Permutations

Permutation means arrangement where order matters.

Number of ways to arrange r items from n distinct items:

$$P(n, r) = n! / (n - r)!$$

Example:

How many ways to choose president and vice president from 5 people?

Order matters.

$$5 * 4 = 20$$

7. Combinations

Combination means selection where order does not matter.

Number of ways to choose r items from n distinct items:

$$C(n, r) = n! / (r! * (n - r)!)$$

Example:

Choose 2 people from 5.

$$C(5, 2) = 10$$

Because AB and BA are the same selection.

8. Permutation vs Combination

Situation	Formula
Order matters	permutation
Order does not matter	combination
Arrange all n items	$n!$
Choose r from n	$C(n, r)$
Assign roles	$P(n, r)$
Select group	$C(n, r)$

Ask:

If I swap two selected items, is it a different outcome?

If yes, permutation.

If no, combination.

9. nCr Formula

$$C(n, r) = n! / (r! * (n-r)!)$$

Also:

$$C(n, r) = C(n, n-r)$$

So use smaller r :

$$r = \min(r, n - r);$$

10. Computing nCr Without Mod

For small n :

```
long long nCr(int n, int r) {  
    if (r < 0 || r > n) {
```

```

        return 0;
    }

    r = min(r, n - r);

    long long result = 1;

    for (int i = 1; i <= r; i++) {
        result = result * (n - r + i) / i;
    }

    return result;
}

```

This avoids computing huge factorials directly.

11. Pascal's Identity

Important recurrence:

$$C(n, r) = C(n - 1, r - 1) + C(n - 1, r)$$

Meaning:

To choose r from n items:

- include n th item: choose $r - 1$ from $n - 1$
- exclude n th item: choose r from $n - 1$

This forms Pascal's Triangle.

12. Full Problem Walkthrough 1 — LeetCode 118: Pascal's Triangle

12.1 Problem Summary

Generate first `numRows` of Pascal's Triangle.

12.2 C++ Code

```

class Solution {
public:
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> triangle;

        for (int row = 0; row < numRows; row++) {
            vector<int> current(row + 1, 1);

            for (int col = 1; col < row; col++) {
                current[col] = triangle[row - 1][col - 1] +
                    triangle[row - 1][col];
            }

            triangle.push_back(current);
        }
    }
};

```

```
    }  
    return triangle;  
}  
};
```

12.3 Complexity

Time:

$O(\text{numRows}^2)$

Space:

$O(\text{numRows}^2)$

13. Full Problem Walkthrough 2 — LeetCode 119: Pascal's Triangle II

13.1 Problem Summary

Return rowIndex-th row of Pascal's Triangle.

13.2 Space Optimized Code

```
class Solution {  
public:  
    vector<int> getRow(int rowIndex) {  
        vector<int> row(rowIndex + 1, 0);  
  
        row[0] = 1;  
  
        for (int i = 1; i <= rowIndex; i++) {  
            for (int j = i; j >= 1; j--) {  
                row[j] += row[j - 1];  
            }  
        }  
  
        return row;  
    }  
};
```

13.3 Why Loop Backward?

To avoid overwriting values needed for the current row.

14. Modular Arithmetic for Counting

Many counting answers are huge.

Use modulo:

$\text{MOD} = 1,000,000,007$

Rules:

```
add = (a + b) % MOD;
sub = (a - b + MOD) % MOD;
mul = 1LL * a * b % MOD;
```

Division requires modular inverse.

15. Precomputing Factorials Modulo MOD

For many nCr queries:

```
const int MOD = 1000000007;

vector<long long> fact;
vector<long long> invFact;
```

Precompute:

```
fact[i] = i! % MOD
```

and inverse factorial:

```
invFact[i] = modular inverse of fact[i]
```

16. Fast Power for Modular Inverse

If MOD is prime:

```
a(-1) = a(MOD - 2) mod MOD
```

Code:

```
long long modPow(long long base, long long exp, long long MOD) {
    long long result = 1;

    while (exp > 0) {
        if (exp & 1) {
            result = result * base % MOD;
        }

        base = base * base % MOD;
        exp >>= 1;
    }

    return result;
}
```

17. nCr Mod Prime Template

```

class Combinatorics {
public:
    static const int MOD = 1000000007;

    vector<long long> fact;
    vector<long long> invFact;

    long long modPow(long long base, long long exp) {
        long long result = 1;

        while (exp > 0) {
            if (exp & 1) {
                result = result * base % MOD;
            }

            base = base * base % MOD;
            exp >>= 1;
        }

        return result;
    }

    Combinatorics(int maxN) {
        fact.assign(maxN + 1, 1);
        invFact.assign(maxN + 1, 1);

        for (int i = 1; i <= maxN; i++) {
            fact[i] = fact[i - 1] * i % MOD;
        }

        invFact[maxN] = modPow(fact[maxN], MOD - 2);

        for (int i = maxN - 1; i >= 0; i--) {
            invFact[i] = invFact[i + 1] * (i + 1) % MOD;
        }
    }

    long long nCr(int n, int r) {
        if (r < 0 || r > n) {
            return 0;
        }

        return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
    }
};

```

18. Full Problem Walkthrough 3 —

LeetCode 62: Unique Paths, Combinatorics

18.1 Problem Summary

Robot moves from top-left to bottom-right in $m \times n$ grid.

Can move only:

right or down

18.2 Counting Insight

Need total moves:

$(m - 1)$ downs
 $(n - 1)$ rights

Total moves:

$m + n - 2$

Choose positions of down moves:

$C(m + n - 2, m - 1)$

or right moves:

$C(m + n - 2, n - 1)$

18.3 C++ Code

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        int total = m + n - 2;
        int choose = min(m - 1, n - 1);

        long long result = 1;

        for (int i = 1; i <= choose; i++) {
            result = result * (total - choose + i) / i;
        }

        return result;
    }
};
```

18.4 Complexity

Time:

$O(\min(m, n))$

Space:

$O(1)$

19. Full Problem Walkthrough 4 — LeetCode 1641: Count Sorted Vowel Strings

19.1 Problem Summary

Count strings of length n made from vowels:

a, e, i, o, u

such that characters are lexicographically non-decreasing.

19.2 Stars and Bars Insight

We need count multisets of size n chosen from 5 vowels.

Formula:

$$C(n + 5 - 1, 5 - 1) = C(n + 4, 4)$$

19.3 C++ Code

```
class Solution {
public:
    int countVowelStrings(int n) {
        return nCr(n + 4, 4);
    }

    int nCr(int n, int r) {
        long long result = 1;

        for (int i = 1; i <= r; i++) {
            result = result * (n - r + i) / i;
        }

        return result;
    }
};
```

19.4 DP Alternative

Also can be solved with DP, but combinatorics is cleaner.

20. Stars and Bars

Stars and bars counts number of non-negative integer solutions to:

$$x_1 + x_2 + \dots + x_k = n$$

Formula:

$$C(n + k - 1, k - 1)$$

Example:

Distribute 5 identical candies among 3 children.

$$x_1 + x_2 + x_3 = 5$$

Answer:

$$C(5 + 3 - 1, 3 - 1) = C(7, 2) = 21$$

21. Stars and Bars With Positive Values

If each x_i must be at least 1:

$$x_1 + x_2 + \dots + x_k = n$$

Let:

$$y_i = x_i - 1$$

Then:

$$y_1 + y_2 + \dots + y_k = n - k$$

Answer:

$$C(n - 1, k - 1)$$

22. Inclusion-Exclusion Principle

Used to count union of overlapping sets.

For two sets:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

For three sets:

$$\begin{aligned} &|A \cup B \cup C| \\ &= |A| + |B| + |C| \\ &- |A \cap B| - |A \cap C| - |B \cap C| \\ &+ |A \cap B \cap C| \end{aligned}$$

Use when direct counting double-counts overlaps.

23. Full Problem Walkthrough 5 — Count Numbers Divisible by 2 or 3

23.1 Problem

Count numbers from 1 to n divisible by 2 or 3.

23.2 Inclusion-Exclusion

```
count divisible by 2 = n / 2
count divisible by 3 = n / 3
count divisible by both = n / lcm(2, 3) = n / 6
```

Answer:

```
n/2 + n/3 - n/6
```

23.3 Code

```
long long countDivisibleBy2Or3(long long n) {
    return n / 2 + n / 3 - n / 6;
}
```

24. Full Problem Walkthrough 6 — LeetCode 1711: Count Good Meals

24.1 Problem Summary

A good meal is pair of dishes whose sum is a power of two.
Return count modulo MOD.

24.2 Counting Insight

For each x , need previous y such that:

```
x + y = powerOfTwo
```

So:

```
y = powerOfTwo - x
```

Use frequency map.

24.3 C++ Code

```
class Solution {
public:
    int countPairs(vector<int>& deliciousness) {
        const int MOD = 1000000007;

        unordered_map<int, int> freq;

        long long answer = 0;

        for (int x : deliciousness) {
            for (int power = 1; power <= (1 << 22); power <= 1) {
                int need = power - x;

                if (freq.count(need)) {
                    answer += freq[need];
                    answer %= MOD;
                }
            }
        }
    }
}
```

```

        freq[x]++;
    }

    return answer;
}
};

```

24.4 Complexity

There are about 22 powers to check.

Time:

$O(22n)$

Space:

$O(n)$

25. Full Problem Walkthrough 7 — LeetCode 1814: Count Nice Pairs in an Array

25.1 Problem Summary

A pair i, j is nice if:

```
nums[i] + rev(nums[j]) == nums[j] + rev(nums[i])
```

Rearrange:

```
nums[i] - rev(nums[i]) == nums[j] - rev(nums[j])
```

So count equal transformed values.

25.2 C++ Code

```

class Solution {
public:
    int reverseNumber(int x) {
        int result = 0;

        while (x > 0) {
            result = result * 10 + x % 10;
            x /= 10;
        }

        return result;
    }

    int countNicePairs(vector<int>& nums) {
        const int MOD = 1000000007;

        unordered_map<int, long long> freq;

```

```

        long long answer = 0;

        for (int x : nums) {
            int key = x - reverseNumber(x);

            answer = (answer + freq[key]) % MOD;

            freq[key]++;
        }

        return answer;
    }
};

```

25.3 Counting Lesson

When pair condition can be transformed into:

```
f(i) == f(j)
```

use frequency counts.

26. Full Problem Walkthrough 8 — LeetCode 1395: Count Number of Teams

26.1 Problem Summary

Given ratings, count teams of 3 soldiers where ratings are strictly increasing or strictly decreasing.

26.2 Counting Around Middle

For each j as middle:

```

leftLess
leftGreater
rightLess
rightGreater

```

Increasing teams:

```
leftLess * rightGreater
```

Decreasing teams:

```
leftGreater * rightLess
```

26.3 C++ Code

```

class Solution {
public:
    int numTeams(vector<int>& rating) {
        int n = rating.size();
    }
};

```

```

    int answer = 0;

    for (int j = 0; j < n; j++) {
        int leftLess = 0;
        int leftGreater = 0;
        int rightLess = 0;
        int rightGreater = 0;

        for (int i = 0; i < j; i++) {
            if (rating[i] < rating[j]) {
                leftLess++;
            } else if (rating[i] > rating[j]) {
                leftGreater++;
            }
        }

        for (int k = j + 1; k < n; k++) {
            if (rating[k] < rating[j]) {
                rightLess++;
            } else if (rating[k] > rating[j]) {
                rightGreater++;
            }
        }

        answer += leftLess * rightGreater;
        answer += leftGreater * rightLess;
    }

    return answer;
}
};

```

26.4 Complexity

Time:

$O(n^2)$

Space:

$O(1)$

27. Counting With Duplicates

If there are duplicate items, permutations reduce.

Example:

How many distinct arrangements of:

AAB

Total permutations if all distinct:

$$3! = 6$$

But A repeats twice, so divide by:

$$2!$$

Answer:

$$3! / 2! = 3$$

Arrangements:

AAB
ABA
BAA

General formula:

If counts are:

$$c_1, c_2, \dots, c_k$$

Total distinct permutations:

$$n! / (c_1! * c_2! * \dots * c_k!)$$

28. Catalan Numbers

Catalan numbers count many recursive structures.

Examples:

- valid parentheses strings
- unique BSTs
- non-crossing handshakes
- triangulations
- ways to parenthesize

Formula:

$$\text{Catalan}(n) = C(2n, n) / (n + 1)$$

First values:

$$1, 1, 2, 5, 14, 42, 132$$

29. Full Problem Walkthrough 9 — Unique Binary Search Trees

29.1 Problem Summary

Given n , count unique BSTs with values $1..n$.

29.2 Catalan / DP

Choose root.

Left subtree has left nodes.

Right subtree has right nodes.

$$dp[n] = \sum dp[\text{left}] * dp[\text{right}]$$

29.3 C++ Code

```
class Solution {
public:
    int numTrees(int n) {
        vector<int> dp(n + 1, 0);

        dp[0] = 1;
        dp[1] = 1;

        for (int nodes = 2; nodes <= n; nodes++) {
            for (int left = 0; left < nodes; left++) {
                int right = nodes - 1 - left;

                dp[nodes] += dp[left] * dp[right];
            }
        }

        return dp[n];
    }
};
```

29.4 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

30. Full Problem Walkthrough 10 — Handshakes That Don't Cross

30.1 Problem Summary

There are even number of people around circle.

Count ways to pair them so handshakes do not cross.

This is Catalan.

If people = $2n$:

answer = Catalan(n)

30.2 DP Recurrence

Pair person 0 with some odd-index person.

This splits circle into two independent parts.

$dp[n] = \text{sum } dp[\text{leftPairs}] * dp[\text{rightPairs}]$

30.3 C++ Code

```

class Solution {
public:
    int numberOfWays(int numPeople) {
        const int MOD = 1000000007;

        int pairs = numPeople / 2;

        vector<long long> dp(pairs + 1, 0);
        dp[0] = 1;

        for (int n = 1; n <= pairs; n++) {
            for (int left = 0; left < n; left++) {
                int right = n - 1 - left;

                dp[n] = (dp[n] + dp[left] * dp[right]) % MOD;
            }
        }

        return dp[pairs];
    }
};

```

31. Derangements Preview

A derangement is a permutation where no element stays in original position.

Number of derangements of n items:

$n!$

Recurrence:

$D[n] = (n - 1) * (D[n - 1] + D[n - 2])$

Base:

$D[0] = 1$

$D[1] = 0$

Example:

For n = 3:

[2,3,1]

[3,1,2]

Answer:

2

32. Full Problem Walkthrough 11 — Number of Ways to Reorder Array to Get Same BST

32.1 Problem Summary

Given nums insertion order into BST.

Count reorderings that produce same BST.

32.2 Counting Insight

Root is first element.

Smaller elements go left.

Larger elements go right.

To preserve same BST:

- left subtree relative order must produce same left BST
- right subtree relative order must produce same right BST
- left and right elements can be interleaved

If left size = L and right size = R:

```
ways = C(L + R, L) * ways(left) * ways(right)
```

32.3 C++ Code

```
class Solution {
public:
    static const int MOD = 1000000007;
    vector<vector<long long>> comb;

    long long dfs(vector<int>& nums) {
        int n = nums.size();

        if (n <= 2) {
            return 1;
        }

        vector<int> left;
        vector<int> right;

        for (int i = 1; i < n; i++) {
            if (nums[i] < nums[0]) {
                left.push_back(nums[i]);
            } else {
                right.push_back(nums[i]);
            }
        }

        long long leftWays = dfs(left);
        long long rightWays = dfs(right);
```

```

        long long mergeWays = comb[n - 1][left.size()];

        return mergeWays * leftWays % MOD * rightWays % MOD;
    }

    int numOfWays(vector<int>& nums) {
        int n = nums.size();

        comb.assign(n + 1, vector<long long>(n + 1, 0));

        for (int i = 0; i <= n; i++) {
            comb[i][0] = 1;
            comb[i][i] = 1;

            for (int j = 1; j < i; j++) {
                comb[i][j] = (comb[i - 1][j - 1] + comb[i - 1][j]) %
MOD;
            }
        }

        return (dfs(nums) - 1 + MOD) % MOD;
    }
};

```

32.4 Why Subtract 1?

Problem asks number of reorderings excluding original order.

33. Common Counting Edge Cases

33.1 $n = 0$

Often:

$$0! = 1$$

$$C(n, 0) = 1$$

33.2 $r > n$

$$C(n, r) = 0$$

33.3 Mod Negative

Use:

$$(x - y + \text{MOD}) \% \text{MOD}$$

33.4 Duplicate Items

Divide by repeated factorials if counting distinct arrangements.

33.5 Overflow

Use long long.

34. Common Counting Mistakes

Mistake 1 — Confusing Permutations and Combinations

Ask if order matters.

Mistake 2 — Double Counting

Use inclusion-exclusion or divide by factorials when duplicates exist.

Mistake 3 — Forgetting Modulo

Large answers need modulo.

Mistake 4 — Wrong nCr Bounds

Return 0 if $r < 0$ or $r > n$.

Mistake 5 — Integer Division Under Mod

Cannot divide normally under modulo.

Need modular inverse.

35. Interview Explanation Templates

35.1 Combination

I use combinations because order does not matter. Choosing r items from n gives $C(n, r)$.

35.2 Permutation

I use permutations because order or role assignment matters.

35.3 Grid Paths

The path consists of fixed number of down and right moves. I choose where one type of move appears among all moves.

35.4 Inclusion-Exclusion

I add counts of individual sets and subtract intersections to correct double counting.

35.5 Catalan

The structure can be split around a chosen pair/root, giving the Catalan recurrence.

36. Complexity Patterns

Operation / Problem	Time	Space
nCr direct	$O(r)$	$O(1)$
Pascal triangle	$O(n^2)$	$O(n^2)$
nCr with precompute	$O(\max N)$ precompute, $O(1)$ query	$O(\max N)$
Unique Paths combinatorics	$O(\min(m, n))$	$O(1)$
Catalan DP	$O(n^2)$	$O(n)$
Count pairs with map	$O(n)$ or $O(n \log n)$	$O(n)$
Reorder BST counting	$O(n^2)$ typical	$O(n^2)$ comb table

37. Expanded Practice Set

37.1 Must-Do Easy / Medium

- 17. Pascal's Triangle
- 18. Pascal's Triangle II
- 19. Unique Paths combinatorics
- 20. Count Sorted Vowel Strings
- 21. Count Good Meals
- 22. Count Nice Pairs
- 23. Count Number of Teams
- 24. Unique Binary Search Trees

37.2 Strong Interview Practice

- 25. Number of Ways to Reorder Array to Get Same BST
- 26. Handshakes That Don't Cross
- 27. Count Ways to Build Rooms in an Ant Colony
- 28. Count Anagrams
- 29. Number of Music Playlists
- 30. Derangements
- 31. Inclusion-exclusion counting problems

37.3 Hard / Stretch

- 32. Catalan advanced applications
 - 33. Stars and bars with upper bounds
 - 34. Inclusion-exclusion with many sets
 - 35. Lucas theorem preview
 - 36. Burnside's Lemma preview
 - 37. Polya counting preview
-

38. Suggested Study Order

Step 1 — Basics

Solve:

- 38. Pascal's Triangle
- 39. Pascal's Triangle II
- 40. Unique Paths

Goal:

Understand nCr and Pascal identity.

Step 2 — Pair Counting

Solve:

- 41. Count Good Meals
- 42. Count Nice Pairs
- 43. Count Number of Teams

Goal:

Transform pair/triple conditions into counts.

Step 3 — Stars and Bars

Solve:

- 44. Count Sorted Vowel Strings
- 45. candy distribution examples

Goal:

Count non-negative solutions.

Step 4 — Catalan

Solve:

- 46. Unique BSTs
- 47. Non-crossing Handshakes

Goal:

Recognize recursive split structures.

Step 5 — Advanced Modular Counting

Solve:

- 48. Reorder Array to Same BST
- 49. Count Anagrams
- 50. Build Rooms

Goal:

Use nCr modulo prime.

39. Pattern Decision Table

Problem Clue	Counting Technique
arrange all items	factorial
choose group	combination
assign ordered roles	permutation
grid paths right/down	$C(\text{total moves, down moves})$
count multisets	stars and bars
union of overlapping conditions	inclusion-exclusion
pair condition $f(i)=f(j)$	frequency map
triples with middle	count left/right options
recursive non-crossing structure	Catalan
same BST reorderings	combine left/right with nCr
duplicate characters	divide by factorial counts
modulo division	modular inverse

40. Mini Problem 1 — Count Pairs From Frequency

If a value appears f times, number of pairs:

```
C(f, 2) = f * (f - 1) / 2
long long countPairsFromFreq(int f) {
    return 1LL * f * (f - 1) / 2;
}
```

41. Mini Problem 2 — Count Anagrams of a Word

For word with length n and character counts c :

```
ways = n! / product(c!)
```

Modulo prime:

```
ways = fact[n];

for each count c:
    ways = ways * invFact[c] % MOD;
```

42. Mini Problem 3 — Number of Binary Strings With k Ones

Length n binary strings with exactly k ones:

```
C(n, k)
```

Because choose positions of ones.

43. Mini Problem 4 — Number of Ways to Put Separators

Split n items into k non-empty groups.

Need $k - 1$ separators among $n - 1$ gaps:

```
C(n - 1, k - 1)
```

44. Debugging Counting Problems

If counting solution fails, check:

51. Does order matter?

- 52. Are items distinct or identical?
 - 53. Are groups labeled or unlabeled?
 - 54. Are duplicates present?
 - 55. Is there overcounting?
 - 56. Should inclusion-exclusion be used?
 - 57. Is modulo division handled with inverse?
 - 58. Are nCr bounds valid?
 - 59. Did you subtract excluded original arrangement if required?
 - 60. Are constraints small enough for DP instead?
-

45. What to Master Before Chapter 52

Before moving to Probability and Randomized Algorithms, make sure you can:

- Explain factorial
 - Explain permutations
 - Explain combinations
 - Compute nCr
 - Use Pascal's identity
 - Use nCr modulo prime
 - Precompute factorial and inverse factorial
 - Use stars and bars
 - Use inclusion-exclusion
 - Count pairs with frequency maps
 - Recognize Catalan patterns
 - Solve Unique Paths combinatorially
 - Solve Count Nice Pairs
 - Solve Count Good Meals
 - Understand same-BST reorder counting
-

46. Final Cheat Sheet

Factorial

$$n! = n * (n - 1) * \dots * 1$$

Permutation

$$P(n, r) = n! / (n-r)!$$

Combination

$$C(n, r) = n! / (r!(n-r)!)$$

Pascal Identity

$$C(n, r) = C(n-1, r-1) + C(n-1, r)$$

Grid Paths

$$C(m+n-2, m-1)$$

Stars and Bars

Non-negative:

$$C(n+k-1, k-1)$$

Positive:

$$C(n-1, k-1)$$

Pair Count

$$C(f, 2) = f(f-1)/2$$

Catalan

$$\text{Catalan}(n) = C(2n, n) / (n+1)$$

Modular nCr

$$\text{fact}[n] * \text{invFact}[r] \% \text{MOD} * \text{invFact}[n-r] \% \text{MOD}$$

47. Core Takeaway

Combinatorics is about counting without brute force.

The first question is always:

Does order matter?

Then ask:

Are items distinct?
Are groups labeled?
Are duplicates present?
Is there overlap causing double-counting?
Is modulo involved?

Core tools:

- factorials for arrangements
- permutations for ordered selections
- combinations for unordered selections
- stars and bars for distributions
- inclusion-exclusion for overlapping sets
- Catalan numbers for recursive non-crossing structures
- frequency maps for pair counting

Counting problems become much easier when you identify the structure before coding.